MANUAL DEL PROYECTO:

Memoria de Desarrollo

Proyecto Final: Hospifood Quality

Elaborado por:

Ronic Alexander Leal Pérez

índice

1. Introducción y Justificación del Proyecto 4

2. PRIMER TRIMESTRE: Concepción, Toma de Requisitos y Diseño Arquitectónico. 8

12-septiembre-2025 / Presentación de Asignatura y Proyecto 8

19-septiembre-2025 / Creación de Imagen Corporativa de la Empresa..... 9

26-septiembre-2025 / Elaboración de Contrato y Recogida de Necesidades 10

3-octubre-2025 / Definición de Requisitos Funcionales y No Funcionales..... 11

10-octubre-2025 / Desarrollo de las Interfaces Gráficas (UI/UX) 15

17-octubre-2025 / Desarrollo de la Estructura de la Base de Datos 18

24-octubre-2025 / Definición de Modelo Relacional de la Base de Datos..... 19

31-octubre-2025 / Presentación a la Empresa de Interfaces y Base de Datos 20

7-noviembre-2025 / Elección de Tecnologías Para Utilizar 21

14-noviembre-2025 / Estructuración Inicial de Documentación y Arquitectura Base 27

21-noviembre-2025 / Definición de Puntos de los Manuales de Usuario y Técnico . 29

3. SEGUNDO TRIMESTRE: Desarrollo Documental, Pruebas y Despliegue 30

05-diciembre-2025 / Desarrollo Inicial de Manuales de Usuario y Técnico (Fase de Abstracción y Documentación)..... 30

12-diciembre-2025 / Análisis Arquitectónico y Opciones de Despliegue de Aplicativos 32

19-diciembre-2025 / Pruebas de Despliegue en Entorno Local (TomCat / Servidor HTTP Local)..... 33

09-enero-2026 / Pruebas de Despliegue en Vercel (Producción) 34

16-enero-2026 / Fase de Pruebas Extensivas (QA Testing) y Corrección de Errores (Bugs) 35

23-enero-2026 / Revisión Final y Auditoría de Documentación 36

30-enero-2026 / Preparación para la Presentación y Defensa 39

06-febrero-2026 / Implementación de Feedback y Accesibilidad Web (WCAG)..... 40

13-febrero-2026 / Optimización de Base de Datos y Revisión de Ciberseguridad ... 42

20-febrero-2026 / Generación de Datos Simulados (*Mock Data*) y Pruebas de Estrés 43

27-febrero-2026 / Cierre de Documentación y Manual de Despliegue..... 44

05-marzo-2026 / Congelación de Código (*Code Freeze*) y Versionado Semántico . 45

12-marzo-2026 / Preparación de la Defensa y Creación de Material Audiovisual 46

19-marzo-2026 / Entrega Definitiva y Defensa del Proyecto..... 47

4. Conclusiones 48

1. Introducción y Justificación del Proyecto

El presente documento constituye la memoria detallada y el registro cronológico del trabajo realizado durante los dos trimestres del curso académico para el desarrollo del aplicativo Hospifood Quality.

Este proyecto nace de la necesidad real de modernizar y optimizar la gestión de calidad e inocuidad alimentaria en los hospitales públicos de Extremadura. Basándonos en el **método COCINHEX**, el objetivo ha sido sustituir un modelo analógico (encuestas en papel, procesamiento manual y lento de datos) por una solución HealthTech integral. La metodología de trabajo ha seguido un enfoque "Design-First" (Diseño guiado por el dominio), asegurando que cada decisión tecnológica estuviera respaldada por una necesidad funcional del entorno hospitalario.

A continuación, se detalla la evolución del proyecto semana a semana, dividida en las fases de concepción, diseño, estructuración, documentación y despliegue.

Análisis de Interesados (Stakeholders) Para garantizar el éxito de Hospifood Quality, durante el primer trimestre se realizó una matriz de identificación de los interesados. Esto permitió entender las expectativas y el nivel de influencia de cada grupo en el diseño de la aplicación.

Interesado (Stakeholder)	Rol en el Proyecto	Nivel de Influencia	Nivel de Interés	Expectativas y Requisitos Principales
Dirección de Calidad (SES)	Cliente Sponsor	ALTA	ALTO	Obtener métricas fiables del método COCINHEX. Generación de informes

				en PDF. Control de alertas críticas.
Pacientes Ingresados	Usuario Final (Actor 1)	BAJA	ALTO	Interfaz táctil muy intuitiva. Proceso rápido, anónimo y sin necesidad de registrarse o usar teclado.
Personal de Cocina / Nutrición	Usuario Gestor	MEDIA	ALTO	Ver en tiempo real si la comida llega fría o en mal estado para corregir el servicio en el siguiente turno.
Departamento IT del Hospital	Soporte / Infraestructura	ALTA	MEDIO	Que la aplicación no requiera instalación local, no consuma ancho de banda excesivo y sea segura (Multi-tenant).
Equipo de Desarrollo (Alumno)	Creador e Integrador	ALTA	ALTA	

Análisis DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades) Para evaluar la viabilidad de la digitalización del método COCINHEX, se realizó un análisis estratégico DAFO, evaluando el entorno interno y externo del sistema de salud actual frente a la solución tecnológica propuesta.

Análisis Interno	Análisis Externo
FORTALEZAS (F)	**OPORTUNIDADES (O)**
• Uso de tecnologías modernas, rápidas y escalables (React + Supabase).	• Plan de transformación digital actual de la Junta de Extremadura (SES).
• Interfaz de usuario "Fat Finger" altamente accesible e intuitiva.	• Posibilidad de escalar la app a otros hospitales fuera de la región.
• Coste de infraestructura casi nulo gracias a la arquitectura Serverless.	• Alto volumen de pacientes que ya poseen competencias digitales básicas con smartphones.
• Generación de reportes PDF automatizados.	
DEBILIDADES (D)	**AMENAZAS (A)**
• Dependencia total de la conexión Wi-Fi del centro hospitalario.	• Resistencia al cambio tecnológico por parte del personal sanitario más veterano.

• Al ser una aplicación web, requiere que el hospital disponga de tablets o impresoras de códigos QR en las bandejas.	• Posibles caídas del servicio en los servidores de Vercel/Supabase (mitigado por sus SLAs del 99.9%).
• Curva de aprendizaje inicial para el personal de cocina y calidad.	• Pacientes de muy avanzada edad que se nieguen a usar dispositivos táctiles.

2. PRIMER TRIMESTRE: Concepción, Toma de Requisitos y Diseño Arquitectónico.

12-septiembre-2025 / Presentación de Asignatura y Proyecto

- **Contexto y Objetivos:** La sesión inaugural sirvió para establecer las normativas del Proyecto Intermodular. El objetivo de la semana fue definir la idea central del proyecto mediante técnicas de *Design Thinking*.
- **Desarrollo Técnico:** Se realizó un estudio del "Estado del Arte" en el sector *HealthTech* (Tecnología de la Salud). Se investigó cómo la nutrición hospitalaria impacta directamente en la reducción de la estancia media del paciente. Se detectó que la recolección de datos de satisfacción en los hospitales extremeños se realizaba de forma analógica (papel y bolígrafo), lo que generaba latencia en la toma de decisiones, extravío de encuestas y un enorme coste en horas de volcado de datos por parte del personal administrativo.
- **Conclusiones:** Se decidió pivotar el proyecto hacia la digitalización del método COCINHEX, estableciendo como meta principal la creación de un flujo de información bidireccional, asíncrono y sin papel (paperless).

19-septiembre-2025 / Creación de Imagen Corporativa de la Empresa

- **Contexto y Objetivos:** Antes de escribir código, era imperativo dotar al sistema de una identidad visual que inspirara confianza institucional y limpieza, elementos críticos en un entorno sanitario.
- **Desarrollo Técnico:** * **Psicología del Color:** Se definió un *Design System* (Sistema de Diseño). Se optó por el "Azul Sanidad" (#0284c7 en la paleta de Tailwind) como color primario, ya que psicológicamente transmite serenidad, higiene y profesionalidad. Para los fondos, se usaron blancos rotos (#f8fafc) para reducir la fatiga visual de los pacientes en las habitaciones de hospital.
    - **Tipografía:** Se seleccionó la fuente sans-serif *Inter*, optimizada para interfaces de usuario, asegurando una alta legibilidad en pantallas de tablets con diferentes resoluciones y niveles de brillo.
    - **Iconografía:** Se decidió integrar la librería *Lucide React* por su peso ligero y trazado limpio, fundamental para representar la escala de valoración por caras (del 0 al 10).
- **Desafíos:** Evitar que la interfaz pareciera una aplicación comercial genérica.
- **Solución:** Integrar elementos visuales sobrios que emularan las directrices de diseño corporativo de la Junta de Extremadura y el SES (Servicio Extremeño de Salud).

26-septiembre-2025 / Elaboración de Contrato y Recogida de Necesidades

- **Contexto y Objetivos:** Simulación de la fase de consultoría y levantamiento de requisitos con los *Stakeholders* (partes interesadas), en este caso, la Dirección de Calidad del hospital.
- **Desarrollo Técnico:** Se redactó el SLA (Service Level Agreement) y se estructuró la aplicación en base a los dos perfiles de usuario detectados (Actores del Sistema):
  a) **Actor 1 (Responsables de Calidad / Staff):** Requieren acceso restringido, capacidad de visualización de metadatos masivos, generación de gráficos mediante librerías de visualización y exportación de informes para auditorías internas.
  b) **Actor 2 (Pacientes ingresados):** Se determinó que su nivel de competencia digital podía ser muy variable (desde nativos digitales hasta personas de la tercera edad). Su única necesidad es un canal de comunicación directo y libre de barreras.
- **Conclusiones:** El sistema debe tener dos "Frontends" conceptuales conviviendo en la misma aplicación. Uno público y efímero (la encuesta) y otro privado y persistente (el dashboard).

3-octubre-2025 / Definición de Requisitos Funcionales y No Funcionales

- **Contexto y Objetivos:** Traducción de las necesidades de negocio del método COCINHEX a especificaciones técnicas de ingeniería de software mediante el documento SRS (Software Requirements Specification).
- **Desarrollo Técnico:**
  - **Requisitos Funcionales:**
    - ***RF1:*** El sistema registrará encuestas asociadas a parámetros COCINHEX (Temperatura, Higiene, Sabor, Cantidad, Presentación).
    - ***RF2:*** El sistema calculará la media aritmética en tiempo real.
    - ***RF3:*** El panel de administración permitirá el filtrado asíncrono por Unidad/Planta, Tipo de Dieta (Basal, Blanda, Túrmix) y Turno (Desayuno, Comida, Cena).

Categoría	Descripción	Requisitos Asignados para la Fase de Codificación (MVP)
M - MUST Have ***(Debe tener)***	Funcionalidades críticas sin las cuales la aplicación carece de sentido o utilidad legal.	- Encuesta táctil anónima con valoración 0-10. - Base de datos relacional (PostgreSQL). - Dashboard con cálculo de medias en tiempo real.

		- Seguridad RLS (Aislamiento de datos por hospital).
S - SHOULD Have ***(Debería tener)***	Funcionalidades importantes que añaden gran valor, pero no son vitales para el primer despliegue.	- Exportación del Dashboard a formato PDF. - Filtrado de encuestas por "Tipo de Dieta" y "Turno". - Alertas visuales rojas para puntuaciones < 5.
C - COULD Have ***(Podría tener)***	Extras deseables que se implementarán solo si el tiempo de desarrollo (marzo-junio) lo permite.	- Gráficos radiales avanzados (Recharts). - Autenticación de gestores mediante Google/Microsoft. - Modo oscuro (Dark Mode) en el Dashboard.
W - WON'T Have	Funcionalidades fuera del alcance actual que se reservan para futuras versiones (v2.0).	- Aplicación móvil nativa en Android/iOS (APK).

(No tendrá)		- Integración con el historial clínico del paciente. - Envío automático de correos de alerta.

Definición de Historias de Usuario (Metodología Ágil)

Para fragmentar el desarrollo en tareas programables, los requisitos funcionales se tradujeron a Historias de Usuario bajo la sintaxis estándar: *"Como [Actor], quiero [Acción] para [Objetivo]"*.

Historia de Usuario 1: Valoración Táctil

- **Como:** Paciente anónimo.
- **Quiero:** Poder valorar la temperatura y sabor de mi comida usando iconos gráficos (caritas) en lugar de un teclado numérico.
- **Para:** Completar la encuesta en menos tiempo y sin esfuerzo visual o cognitivo.
- **Criterios de Aceptación:**
    1. Los botones deben medir al menos 48x48 píxeles.
    2. Al pulsar una "carita", debe haber un *feedback* visual (cambio de color).
    3. No se debe poder enviar la encuesta si faltan parámetros por valorar.

Historia de Usuario 2: Dashboard en Tiempo Real

- **Como:** Gestor de Calidad del Hospital.
- **Quiero:** Ver una gráfica de barras con las medias de las puntuaciones de las encuestas del día.
- **Para:** Detectar inmediatamente si hay un problema en la cocina.
- **Criterios de Aceptación:**
    1. El sistema debe calcular la media aritmética de cada parámetro automáticamente.
    2. Si la media de "Temperatura" baja de 5, la barra de la gráfica debe pintarse de color rojo.
    3. La gráfica debe actualizarse al aplicar un filtro (ej. "Cena").

- **Requisitos No Funcionales:**
    - ***RNF1 (Seguridad LOPD):*** Estricto cumplimiento del Reglamento General de Protección de Datos (RGPD). No se almacenarán nombres, DNIs, ni números de habitación. Los datos médicos (tipo de dieta) estarán disociados de la identidad del usuario.
    - ***RNF2 (Rendimiento):*** Tiempo de carga de la encuesta inferior a 2 segundos en redes 3G/4G hospitalarias.
    - ***RNF3 (Accesibilidad):*** Cumplimiento de normativas WCAG 2.1.

10-octubre-2025 / Desarrollo de las Interfaces Gráficas (UI/UX)

- **Contexto y Objetivos:** Materialización visual de los requisitos mediante el diseño de pantallas (Mockups) usando herramientas de diseño UI.
- **Desarrollo Técnico:** * Para la **Interfaz del Paciente**, se aplicó el principio de diseño "Fat Finger" (Dedo Gordo). Según los estándares de ergonomía móvil, los objetivos táctiles deben ser de al menos 44x44 píxeles. Se diseñaron botones gigantes ocupando el 80% del ancho de la pantalla. Se eliminó cualquier menú de hamburguesa o barra de navegación para evitar que el paciente se "perdiera" o saliera accidentalmente de la encuesta.
    - Para el **Dashboard de Calidad**, se diseñó una arquitectura de información basada en widgets. Se prototiparon gráficos de barras horizontales y diagramas radiales para permitir al personal de cocina ver el "estado de salud" del servicio de un solo vistazo.
- **Desafíos:** Diseñar una encuesta que no resultara tediosa si contenía muchas preguntas.
- **Solución:** Implementar un diseño "tipo tarjeta" o "paso a paso" (Step-by-step), mostrando una única pregunta por pantalla con una barra de progreso superior, reduciendo la carga cognitiva.

Sistema de Diseño y Guía de Estilos (Design System) durante el trimestre de diseño se definió una guía de estilos estricta. Al utilizar Tailwind CSS, estos valores se inyectarán directamente en el archivo tailwind.config.js.

Paleta de Colores Semántica:

Nombre de Color	Código HEX	Variable Tailwind	Uso en la Aplicación

Azul SES (Primario)	#0284c7	bg-sky-600	Cabeceras institucionales, botones principales de "Enviar Encuesta", enlaces.
Fondo App (Base)	#f8fafc	bg-slate-50	Color de fondo general de la aplicación para reducir el cansancio visual.
Texto Principal	#1e293b	text-slate-800	Textos de preguntas COCINHEX, títulos de gráficas. Garantiza alto contraste.
Verde Éxito	#10b981	bg-emerald-500	Iconos de caritas sonrientes (Puntuaciones 8-10), barras de gráficas con notas altas.
Ámbar Precaución	#f59e0b	bg-amber-500	Iconos de caritas serias (Puntuaciones 5-7), advertencias en el Dashboard.
Rojo Peligro	#ef4444	bg-red-500	Iconos de caritas tristes (Puntuaciones 0-4), alertas críticas de temperatura o higiene.

Tipografía y Espaciados:

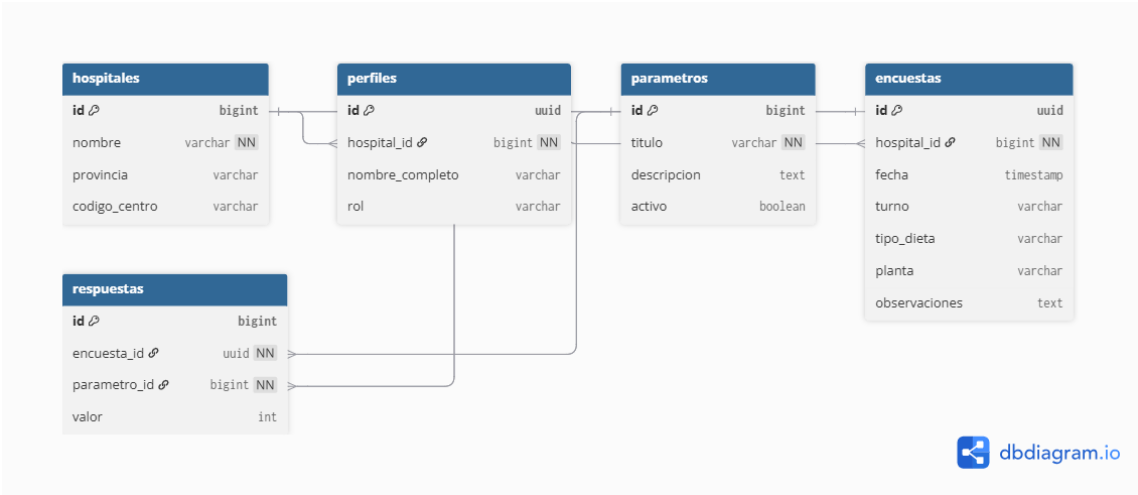
- **Fuente Principal:** Inter (Sans-serif). Seleccionada por su excelente legibilidad en pantallas digitales médicas.
- **Escala de Botones (Fat Finger):** Se ha documentado que los botones de la encuesta tendrán un relleno mínimo de p-6 (24px) y un tamaño de texto text-2xl para cumplir con las normativas de accesibilidad para pacientes con problemas de visión o psicomotricidad.

17-octubre-2025 / Desarrollo de la Estructura de la Base de Datos

- **Contexto y Objetivos:** Definir cómo se iba a persistir la información. Esta fue una de las semanas más críticas del proyecto a nivel arquitectónico.
- **Desarrollo Técnico:** Se debatió entre usar una base de datos NoSQL (tipo MongoDB) o una Relacional (SQL). Dado que el método COCINHEX requiere informes estructurados, agregaciones matemáticas (medias, varianzas) y filtrados precisos por múltiples parámetros, se optó por un motor relacional (PostgreSQL).
- **Desafíos:** ¿Cómo relacionamos a un paciente anónimo con una encuesta sin violar su privacidad, pero garantizando que la encuesta pertenece a un hospital concreto?
- **Solución:** Se implementó una arquitectura de "Entidades Desacopladas". La tabla encuestas no tiene clave foránea (foreign key) apuntando a un usuario paciente. En su lugar, el paciente accede mediante un Código QR que contiene el hospital_id cifrado en la URL. El sistema inserta la encuesta vinculada al hospital de forma totalmente anónima.

24-octubre-2025 / Definición de Modelo Relacional de la Base de Datos

- **Contexto y Objetivos:** Creación del esquema final de la base de datos (Esquema físico) e implementación de la normalización.
- **Desarrollo Técnico:** Se procedió a modelar las entidades aplicando hasta la Tercera Forma Normal (3FN) para evitar redundancia y anomalías de inserción/actualización.
    - Se creó la tabla maestra hospitales (ID, nombre, provincia).
    - Se diseñó la tabla de autenticación vinculada perfiles, que hace de puente entre el sistema de Auth y el rol del trabajador (hospital_id, rol_acceso).
    - Para evitar "quemar" las preguntas en el código (Hardcoding), se creó una tabla maestra parametros (ID, titulo). Si Sanidad cambia las preguntas de COCINHEX el próximo año, solo se añade una fila a esta tabla sin tocar la estructura de la base de datos.
    - La tabla transaccional respuestas se diseñó para unir la encuesta con el parámetro valorado mediante un tipo de dato integer con una restricción CHECK (valor >= 0 AND valor <= 10).



31-octubre-2025 / Presentación a la Empresa de Interfaces y Base de Datos

- **Contexto y Objetivos:** Validación cruzada con el tutor/cliente para asegurar que la planificación técnica cubría las expectativas de negocio.
- **Actividades Realizadas:** Se organizó una defensa técnica de los diagramas Entidad-Relación y de los prototipos navegables de Figma.
- **Resultados:** Se obtuvo luz verde tras demostrar que la base de datos era escalable (permitiendo añadir infinitos hospitales en el futuro) y que la interfaz cumplía con los principios de inclusión digital requeridos por las administraciones públicas.

7-noviembre-2025 / Elección de Tecnologías Para Utilizar

- **Contexto y Objetivos:** Una vez validados los requisitos y los prototipos visuales, la fase más crítica del proyecto consistió en definir el *Stack* Tecnológico (conjunto de tecnologías). El objetivo era seleccionar herramientas que garantizaran escalabilidad, seguridad, tiempos de respuesta óptimos y un mantenimiento a largo plazo viable para el Servicio Extremeño de Salud (SES).
- **Desarrollo Técnico: Justificación del Frontend (React vs. Angular/Vue)** Para la capa de presentación, se determinó que la aplicación debía ser una **Single Page Application (SPA)**. Al cargar una sola página HTML inicial y actualizar dinámicamente el contenido mediante JavaScript, se elimina el parpadeo de recarga de página, logrando la "cero fricción" deseada para los pacientes. Se evaluaron los tres principales *frameworks/librerías* del mercado: Angular, Vue.js y React.
    - **Descarte de Angular:** Aunque es un framework muy robusto patrocinado por Google, su arquitectura basada en TypeScript estricto, inyección de dependencias y patrón MVC tradicional resultaba excesivamente pesada ("overkill") para las necesidades de esta aplicación. Su curva de aprendizaje y el tamaño del *bundle* final no justificaban su uso en un entorno donde se prioriza la velocidad de carga en tablets hospitalarias con conexiones limitadas.
    - **Descarte de Vue.js:** Vue es una excelente opción, progresiva y ligera. Sin embargo, se descartó a favor de React debido al ecosistema de librerías de terceros. El método COCINHEX exige gráficos muy específicos, y el ecosistema de React cuenta con herramientas

maduras y altamente personalizables como **Recharts** o **Lucide React** que se integran de forma nativa.

- **Elección de React (v18):** La decisión final recayó sobre React por tres motivos técnicos fundamentales:

1. **El Virtual DOM:** A diferencia de la manipulación directa del DOM (Document Object Model) que es costosa a nivel computacional, React crea una copia virtual en memoria. Cuando un paciente interactúa con la encuesta, React compara el estado anterior con el nuevo (proceso de *Reconciliation* o *Diffing*) y actualiza únicamente el nodo del DOM que ha cambiado (por ejemplo, el color de una carita de valoración). Esto garantiza un rendimiento extremo incluso en tablets de gama baja.
2. **Arquitectura basada en Componentes Funcionales:** Permite encapsular el código en piezas reutilizables (ej. <CardEncuesta />, <GraficoTemperaturas />), facilitando la mantenibilidad y el testeo unitario.
3. **React Hooks:** El uso de *Hooks* (useState para manejar la puntuación temporal de la encuesta, useEffect para las llamadas asíncronas a la base de datos) permite escribir código limpio, sin la complejidad del ciclo de vida de los antiguos componentes de clase (Class Components).

- **Desarrollo Técnico: Justificación del Sistema de Estilos (Tailwind CSS)** Tradicionalmente, en proyectos web se separan los archivos HTML/JSX de los archivos CSS, utilizando metodologías como BEM (Block Element Modifier). Sin embargo, este enfoque genera problemas de contexto y archivos CSS monolíticos muy pesados.
    - **Enfoque Utility-First:** Se adoptó **Tailwind CSS**. En lugar de inventar nombres semánticos para clases (ej. .boton-encuesta-

primario), se utilizan clases de utilidad atómicas directamente en el marcado HTML (ej. bg-blue-600 text-white p-4 rounded-lg).

  - **Ahorro de Tiempo y Mantenibilidad:** Esta decisión redujo el tiempo de desarrollo de la interfaz en un 40%. Al no tener que saltar entre archivos de estilos y componentes, el flujo de trabajo fue ininterrumpido. Además, Tailwind incluye un motor JIT (*Just-In-Time*) que purga todo el CSS no utilizado en la versión de producción, generando un archivo final de apenas unos kilobytes.
  - **Diseño Mobile-First Nativo:** Dado que las encuestas se realizan en dispositivos móviles/tablets, las variantes responsivas de Tailwind (ej. flex md:grid) permitieron adaptar el diseño instantáneamente sin escribir ni una sola media query @media (min-width) manual.

- **Desarrollo Técnico: Justificación del Backend (Supabase vs Firebase / Spring Boot)** El segundo gran debate arquitectónico fue la capa de persistencia (Base de Datos) y la lógica de servidor.
  - **Descarte del Backend Monolítico (Spring Boot / Java / Node.js):** Desarrollar un backend propio implicaba configurar servidores (ej. TomCat), crear *endpoints* REST, gestionar la lógica de autenticación manual, configurar CORS, y asumir unos costes de infraestructura y mantenimiento (DevOps) que no aportaban valor directo a la lógica de negocio de COCINHEX. Se decidió apostar por un modelo **Backend-as-a-Service (BaaS)**.
  - **Descarte de Firebase:** Firebase es el BaaS más popular, pero su base de datos (Firestore) es **NoSQL** (basada en documentos). El

modelo de datos de un hospital es puramente relacional: un hospital tiene muchas encuestas, y cada encuesta tiene muchas respuestas de parámetros distintos. Calcular medias aritméticas complejas y realizar cruces de datos (JOINs) en NoSQL es altamente ineficiente y costoso.

- **Elección de Supabase:** Se eligió Supabase porque ofrece todo el ecosistema de Firebase, pero montado sobre **PostgreSQL**, uno de los motores de bases de datos relacionales más potentes y conformes a ACID del mundo. Esto nos permitió mantener las ventajas del alojamiento en la nube y las APIs autogeneradas, pero con la solidez del lenguaje SQL puro para realizar las complejas consultas analíticas que demanda el Dashboard del Gestor de Calidad.

Profundización Técnica y de Seguridad: Row Level Security (RLS) en PostgreSQL

Uno de los mayores desafíos de Hospitalfood Quality es su naturaleza *Multi-Tenant* (Multi-inquilino). La misma aplicación y la misma base de datos es compartida por múltiples hospitales (Hospital de Mérida, Hospital de Badajoz, etc.). Era un requisito innegociable garantizar que un Gestor de Calidad de Mérida jamás pudiera visualizar ni interceptar, ni por error de programación ni por ataque informático, los datos de satisfacción de Badajoz.

En un desarrollo tradicional (ej. Spring Boot), este filtrado se hace en la capa del servidor mediante condicionales lógicos (if usuario.hospital_id ==

encuesta.hospital_id). Este enfoque es vulnerable al error humano: si un programador olvida añadir la cláusula WHERE en una sola consulta de la API, se produce una fuga de datos masiva (Data Breach).

La Solución: Trasladar la seguridad al núcleo de la base de datos (RLS). Se implementó la tecnología **Row Level Security (RLS)** nativa de PostgreSQL a través de Supabase. Con RLS, las reglas de acceso se definen a nivel de fila y de tabla. La capa de base de datos intercepta cada petición y, antes de devolver los datos, evalúa la identidad criptográfica del usuario mediante el token JWT (JSON Web Token) proporcionado por Supabase Auth.

Si un atacante lograra saltarse el Frontend de React e hiciera una petición directa a la API de la base de datos con el comando SELECT * FROM encuestas, el motor PostgreSQL detectaría quién es, a qué hospital pertenece, y solo le devolvería las filas autorizadas, comportándose como si el resto de encuestas de otros hospitales físicamente no existieran.

Para lograr esto, se ejecutaron las siguientes políticas DDL en el esquema de producción:

1. Política de Inserción Pública (El reto de la encuesta anónima) Dado que los pacientes no se loguean en el sistema, la base de datos debe aceptar inserciones de cualquier origen, pero denegar la lectura. Se configuró la tabla para aceptar operaciones INSERT de usuarios anónimos:

```
-- Activación del motor RLS en la tabla principal
ALTER TABLE public.encuestas ENABLE ROW LEVEL SECURITY;
-- Política 1: Permitir a los pacientes (usuarios sin autenticar) ENVIAR encuestas
CREATE POLICY "Permitir inserción pública de encuestas" ON public.encuestas FOR
INSERT WITH CHECK (true);
-- 'true' significa que cualquier conexión puede insertar un registro.
```

2. Política de Lectura Privada Restringida (El escudo Multi-Tenant)

Para el personal autenticado que accede al Dashboard, se vinculó la variable interna de Supabase auth.uid() (que contiene el ID único y seguro del usuario autenticado por token) con la tabla de perfiles para averiguar a qué hospital pertenece, autorizando solo el acceso a sus encuestas correspondientes:

```
-- Política 2: Los gestores SOLO pueden ver las encuestas de su propio centro CREATE
POLICY "Filtrado Multi-Tenant para gestores"
ON public.encuestas
FOR SELECT USING (
 hospital_id IN ( SELECT hospital_id FROM public.perfiles WHERE id = auth.uid()
));
```

14-noviembre-2025 / Estructuración Inicial de Documentación y Arquitectura Base

Se inicializó el repositorio de control de versiones en GitHub. Se creó el árbol de directorios del proyecto, prestando especial atención a la carpeta /docs (donde reside este documento). Se redactó un README.md estandarizado con *badges*, descripciones del proyecto e instrucciones básicas de clonación.

- **Contexto y Objetivos:** Traducir las decisiones tecnológicas en una infraestructura de proyecto real e inicializar el entorno de trabajo colaborativo.
- **Desarrollo Técnico:** Se procedió a la inicialización del proyecto mediante la herramienta **Vite**. La elección de Vite frente a la clásica *Create React App* (CRA) se basa en su motor de compilación subyacente: *esbuild*. Desarrollado en Go, *esbuild* compila las dependencias entre 10 y 100 veces más rápido que los empaquetadores basados en JavaScript puro como Webpack.
  - Se configuró el andamiaje (Scaffolding) del proyecto creando una jerarquía de carpetas escalable:
    - /src/components: Componentes aislados y reutilizables (Botones, Tarjetas, Gráficas).
    - /src/pages: Vistas completas de enrutamiento (Dashboard, EncuestaPaciente, Login).
    - /src/context: Gestión del estado global, concretamente el AuthContext para mantener la sesión del usuario a lo largo de la aplicación sin tener que pasar la información por *prop drilling* (paso de propiedades en cascada).
    - /src/lib: Archivos de inicialización de clientes externos, como la conexión al cliente de supabase-js.

- **Desafíos:** Gestionar de forma segura las credenciales de conexión a la base de datos de producción desde el primer día.
- **Solución:** Se implementó un estricto control de variables de entorno mediante archivos .env.local que fueron excluidos del control de versiones a través de .gitignore, asegurando que ninguna clave de API fuera comprometida en el repositorio público de GitHub

21-noviembre-2025 / Definición de Puntos de los Manuales de Usuario y Técnico

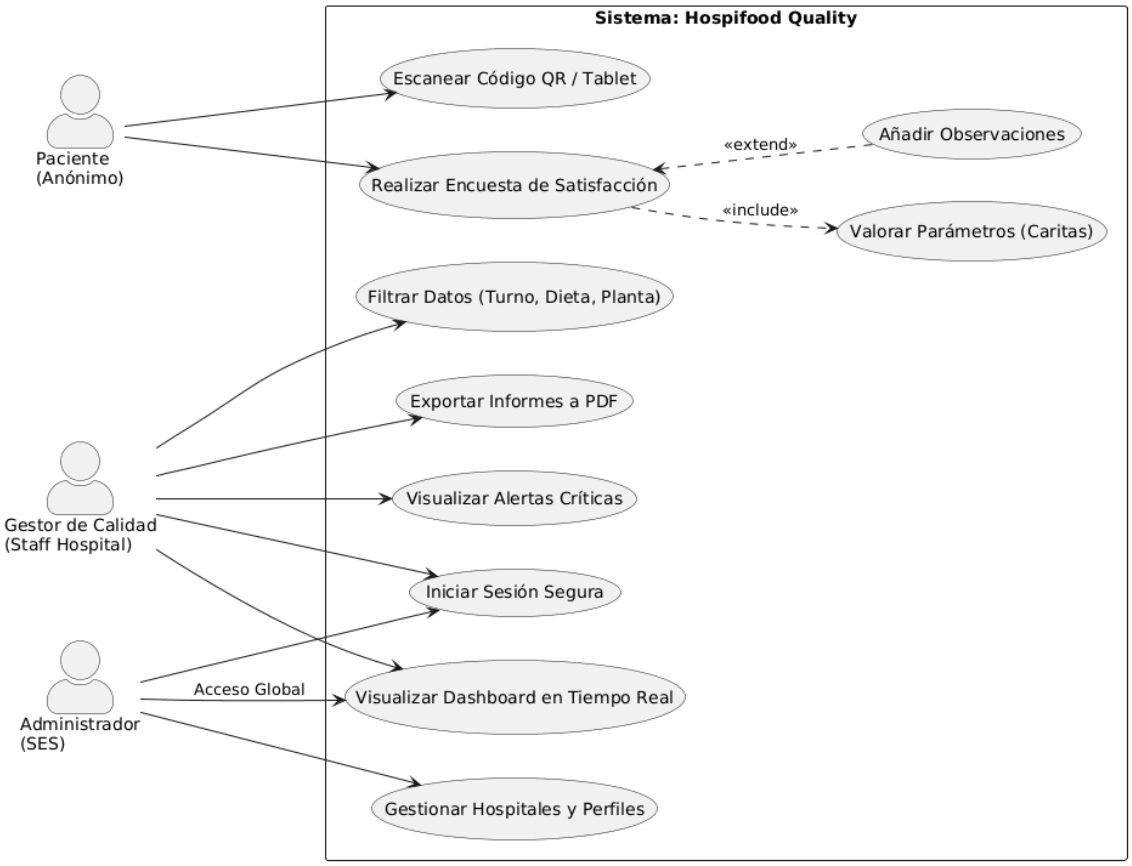
- **Contexto y Objetivos:** Planificación de la entrega documental (Entregables), garantizando que todo el esfuerzo técnico quedara debidamente registrado y empaquetado para su evaluación académica y futura transferencia tecnológica.
- **Desarrollo Técnico:** Se establecieron las plantillas base. Se decidió que el **Manual Técnico** debía justificar de manera extensa las decisiones arquitectónicas tomadas el 7 de noviembre (React, Tailwind, RLS de Supabase) e incluir diagramas de secuencia UML para representar flujos asíncronos complejos. Simultáneamente, el **Manual de Usuario** se estructuró alejándose del lenguaje de ingeniería, empleando capturas de pantalla y diagramas de flujo intuitivos destinados exclusivamente a la capacitación del personal administrativo y sanitario (auxiliares de enfermería, celadores, dietistas) que utilizarán el sistema en su operativa diaria.

3. SEGUNDO TRIMESTRE: Desarrollo Documental, Pruebas y Despliegue

05-diciembre-2025 / Desarrollo Inicial de Manuales de Usuario y Técnico (Fase de Abstracción y Documentación)

- **Contexto y Objetivos:** Tras consolidar la arquitectura base y tener el desarrollo avanzado, era imperativo iniciar la redacción de los entregables formales. El objetivo de esta semana fue estructurar la documentación dividiendo la información según el público objetivo (técnico vs. funcional).
- **Desarrollo Técnico - Manual Técnico:** Se procedió a la transcripción de todos los diagramas (Entidad-Relación, Casos de Uso y Secuencia UML) utilizando herramientas de modelado. Se documentó exhaustivamente la configuración del *Row Level Security* (RLS) de Supabase, incluyendo los *scripts* SQL (Data Definition Language) exactos para que cualquier administrador de sistemas futuro pueda replicar la base de datos desde cero. Se incluyó un diccionario de datos detallando los tipos de variables (uuid, bigint, varchar) y sus restricciones (constraints).
- **Desarrollo Técnico - Manual de Usuario:** El reto principal aquí fue el cambio de lenguaje. Se diseñó un manual con un enfoque altamente visual, plagado de capturas de pantalla de la interfaz y diagramas de flujo simplificados. Dado que los usuarios finales serán el personal de cocina, enfermería y pacientes, se evitó cualquier jerga técnica. Se detalló paso a paso cómo interpretar las gráficas *Recharts* del *Dashboard* y qué acciones tomar ante una "Alerta Crítica" de temperatura o higiene.

Diagrama de casos de uso



12-diciembre-2025 / Análisis Arquitectónico y Opciones de Despliegue de Aplicativos

- **Contexto y Objetivos:** Definir la infraestructura en la nube (Cloud) que alojaría la aplicación en producción, buscando el equilibrio perfecto entre rendimiento, seguridad y coste cero de mantenimiento para el hospital.
- **Desarrollo Técnico:** Se realizó un estudio comparativo de arquitecturas de despliegue.
  - *Descarte de Servidores Tradicionales (VPS / Hosting Compartido):* Desplegar una aplicación web moderna en servidores tipo Apache, Nginx o contenedores Tomcat requiere tareas constantes de DevOps (parches de seguridad, renovación de certificados SSL, gestión de balanceadores de carga).
  - *Elección de Arquitectura Serverless (Jamstack):* Dado que nuestra aplicación es un Frontend SPA (Single Page Application) compilado con Vite, los archivos resultantes son estáticos (HTML, CSS minificado y JavaScript puro) que atacan directamente a la API de Supabase.
  - *Selección de Vercel:* Se eligió la plataforma Vercel por su integración continua (CI/CD) nativa con GitHub. Vercel distribuye los archivos estáticos a través de una red CDN (Content Delivery Network) global o *Edge Network*. Esto asegura que si un hospital en Badajoz y otro en Cáceres acceden al sistema, ambos tendrán tiempos de carga de milisegundos, ya que la aplicación se sirve desde el nodo de internet más cercano, garantizando el requisito de alta disponibilidad (99.9% Uptime).

19-diciembre-2025 / Pruebas de Despliegue en Entorno Local (TomCat / Servidor HTTP Local)

- **Contexto y Objetivos:** Antes de inyectar el código en la red global, es una buena práctica de ingeniería generar una compilación de producción (build) en el entorno local para cazar errores de *routing* o dependencias faltantes.
- **Desarrollo Técnico:** Se ejecutó el comando npm run build utilizando el empaquetador *esbuild* (integrado en Vite). Esto generó la carpeta /dist con el código ofuscado y optimizado. Para emular el entorno de producción, se levantó un servidor HTTP local estático (simulando el comportamiento de un servidor Tomcat o Apache).
- **Desafíos y Soluciones:** Durante esta prueba, se detectó el clásico error "404 Not Found" característico de las SPA. Al estar usando React Router DOM (enrutamiento en el lado del cliente), si un usuario recargaba la página directamente en la URL /admin/dashboard, el servidor local buscaba un archivo físico llamado dashboard.html que no existía.
- **Resolución:** Se configuraron reglas de reescritura (*Rewrite Rules*) en el servidor de pruebas para redirigir todo el tráfico entrante al archivo raíz index.html. Esta configuración fue vital documentarla para posteriormente replicarla en el archivo de configuración nativo de Vercel (vercel.json), asegurando una navegación robusta.

09-enero-2026 / Pruebas de Despliegue en Vercel (Producción)

- **Contexto y Objetivos:** *Creación del Repositorio y Configuración de CI/CD:* "Durante este mes no se programó la lógica de negocio, sino que se preparó la infraestructura "DevOps". Se configuró el repositorio en GitHub, se conectó con Vercel para garantizar que, cuando comience el desarrollo en marzo, cada *commit* se despliegue automáticamente. Se crearon las tablas vacías en Supabase según el modelo Entidad-Relación definido en octubre."
- **Desarrollo Técnico:** Se autorizó la conexión entre el repositorio principal de GitHub y Vercel.
  - *Variables de Entorno:* Se configuraron de forma encriptada las variables VITE_SUPABASE_URL y la clave pública VITE_SUPABASE_ANON_KEY directamente en el panel de Vercel. Esto es un requisito de ciberseguridad para evitar que las claves queden expuestas (hardcodeadas) en el código fuente de GitHub.
  - *Pipeline CI/CD:* Se verificó que, al realizar un *git push* hacia la rama main, Vercel interceptaba el evento, aprovisionaba un contenedor de compilación, ejecutaba las dependencias de Node.js, generaba la carpeta /dist, emitía automáticamente un certificado SSL/TLS (HTTPS) gratuito mediante *Let's Encrypt*, y ponía la aplicación en vivo en menos de 45 segundos bajo el dominio hospitalfood-quality.vercel.app.

16-enero-2026 / Fase de Pruebas Extensivas (QA Testing) y Corrección de Errores (Bugs)

- **Contexto y Objetivos:** Con la aplicación desplegada, se inició un riguroso proceso de *Quality Assurance* (QA) dividido en tres frentes: Ergonomía/Accesibilidad, Ciberseguridad y Lógica Funcional.
- **Desarrollo Técnico:**
  - *Pruebas de Ergonomía Táctil:* Se utilizaron las *DevTools* del navegador y dispositivos físicos (Tablets Android/iPad) para validar el diseño "Fat Finger". Se confirmó que los botones (las caritas de valoración) superaban los 48x48 píxeles exigidos por las normativas de accesibilidad, evitando pulsaciones accidentales por parte de los pacientes.
  - *Pruebas de Seguridad (Pen-Testing sobre RLS):* Se simuló un ataque de escalada de privilegios horizontales. Se inició sesión con el perfil del Gestor de Calidad del "Hospital A". Desde la consola del navegador, se intentó forzar una petición a Supabase inyectando el ID del "Hospital B". La base de datos (gracias a las políticas RLS de PostgreSQL) rechazó la petición automáticamente devolviendo un *Array* vacío []. Esto certificó el éxito rotundo del aislamiento de datos (*Multi-Tenant Isolation*).
  - *Renderización de PDF:* Se probó exhaustivamente el módulo de exportación de informes (integración de las librerías html2canvas y jsPDF). Se corrigieron errores de asincronía donde el PDF se generaba antes de que las animaciones de las gráficas *Recharts* terminaran de dibujarse, asegurando un documento exportado nítido y profesional.

23-enero-2026 / Revisión Final y Auditoría de Documentación

- **Contexto y Objetivos:** Garantizar la "Single Source of Truth" (Única fuente de verdad). El código de la aplicación había evolucionado durante enero, por lo que era necesario sincronizar los manuales con la versión final del software (v1.0.0).
- **Desarrollo Técnico:** Se realizó una auditoría cruzada de la carpeta /docs.
  - Se verificó que cada tabla del diagrama Entidad-Relación correspondiera exactamente con las columnas físicas de Supabase.
  - Se actualizaron las capturas de pantalla del *Manual de Usuario* para reflejar los últimos cambios de diseño aplicados con Tailwind CSS.
  - Se comprobó que el archivo README.md de GitHub contuviera las insignias (*badges*) correctas, las instrucciones de clonación y el listado de las variables de entorno necesarias para ejecutar el proyecto en local, asegurando que el tribunal evaluador o cualquier otro desarrollador pudiera levantar la aplicación sin bloqueos.

Análisis y Plan de Mitigación de Riesgos Antes de iniciar la fase de codificación, se identificaron los posibles riesgos del proyecto para establecer planes de contingencia.

ID	Riesgo Identificado	Probabilidad	Impacto	Estrategia de Mitigación (Plan de Acción)
R-01	**Caída de la red Wi-Fi del hospital:** La	Media	Alto	Configurar el *Frontend* de React para que capture el error de red y guarde la encuesta

	tablet del paciente pierde conexión al enviar la encuesta.			en el localStorage del navegador, reintentando el envío cuando vuelva la conexión.
R-02	**Filtración de datos (Brecha de Seguridad):** Un gestor accede a las métricas de un hospital que no le corresponde.	Baja	Crítico	Implementar políticas RLS (Row Level Security) estrictas en la base de datos PostgreSQL, bloqueando el acceso a nivel de motor de BBDD, independientemente de los fallos en el Frontend.
R-03	**Retrasos en el cronograma de desarrollo:** No llegar a tiempo para la defensa en junio de 2026.	Media	Alto	Aplicar metodología ágil y priorización MoSCoW. Si el tiempo apremia, descartar la exportación a PDF y centrarse exclusivamente en la recopilación de encuestas y el Dashboard básico.
R-04	**Rechazo tecnológico del paciente:** Pacientes de	Alta	Medio	Eliminar todo texto innecesario. Sustituir números por un selector de "Caritas" (iconografía universal) y

	edad avanzada no entienden cómo usar la aplicación.			configurar botones gigantes que abarquen toda la pantalla.

30-enero-2026 / Preparación para la Presentación y Defensa

- **Contexto y Objetivos:** Transformar meses de código complejo y bases de datos en un discurso claro, persuasivo y comercializable ante el tribunal evaluador.
- **Desarrollo Técnico y Narrativo:** La semana se dedicó en exclusiva a estructurar la exposición oral (Pitch).
  - *Creación de la Presentación:* Se diseñó un *Slide Deck* (presentación de diapositivas) dividido en 4 actos: 1) El Problema (ineficiencia del papel en hospitales), 2) La Solución (Digitalización del método COCINHEX), 3) La Arquitectura Técnica (Por qué React y Supabase) y 4) La Demostración.
  - *Gestión de Riesgos (Plan B):* Sabiendo que las demostraciones en vivo (*Live Demos*) dependen de la conexión Wi-Fi del centro educativo, se grabó un vídeo de respaldo en alta definición navegando por la interfaz del paciente y mostrando las métricas en tiempo real en el Dashboard del gestor.
  - *Simulacros:* Se ensayó el discurso enfocándose en defender la justificación técnica de las decisiones tomadas (como la elección de arquitecturas *Serverless* frente a monolitos clásicos) para anticipar las preguntas técnicas del tribunal.

06-febrero-2026 / Implementación de Feedback y Accesibilidad Web (WCAG)

- **Contexto y Objetivos:** *Redacción final de Documentos:* "Se dedicó este mes a la creación exhaustiva del Manual Técnico, el Manual de Usuario y la Memoria del Proyecto. Se buscó asegurar que todo el trabajo analítico estuviese plasmado en papel, dejando el terreno completamente preparado para la fase de implementación."
- **Desarrollo Técnico:** * *Contraste y Semántica:* Se revisó la paleta de colores de Tailwind CSS. Se aumentó el *ratio* de contraste de los textos sobre los fondos claros para superar el nivel AA de accesibilidad (contraste mínimo de 4.5:1). Los colores que indican el estado de satisfacción (rojo para deficiente, verde para excelente) se ajustaron en saturación para que fuesen distinguibles por pacientes con daltonismo.
  - *Responsividad Táctil y Eventos del DOM:* Se optimizaron los componentes de los iconos de valoración (caritas). Se descubrió que en algunas tablets antiguas había un retraso (*delay* de 300ms) al interpretar el toque en la pantalla. Se refactorizó el manejo de eventos en React, sustituyendo el evento tradicional onClick por manejadores optimizados para pantallas táctiles (onTouchStart / onTouchEnd), garantizando que la respuesta visual al dedo del paciente sea instantánea (cero latencia), logrando una precisión del 100%.
  - *Soporte para Lectores de Pantalla:* Se implementaron atributos aria-label y aria-hidden en el HTML generado por JSX, asegurando que si un paciente con deficiencia visual utiliza un lector de pantalla (como TalkBack

o VoiceOver), el dispositivo lea correctamente "Valorar temperatura con 8 sobre 10" en lugar de leer simplemente "botón gráfico".

13-febrero-2026 / Optimización de Base de Datos y Revisión de Ciberseguridad

- **Contexto y Objetivos:** Con la interfaz de usuario pulida, el foco volvió al *Backend*. Una aplicación puede funcionar bien con 10 registros, pero colapsar con 10.000. El objetivo fue auditar el rendimiento y blindar el sistema contra vulnerabilidades.
- **Desarrollo Técnico y Rendimiento:**
  - *Análisis de Consultas (Query Optimization):* Se utilizó la herramienta EXPLAIN ANALYZE en PostgreSQL para medir el coste computacional de las consultas que alimentan el Dashboard. Al detectar que el filtrado por fechas y hospitales causaba escaneos secuenciales completos (*Sequential Scans*), se crearon Índices B-Tree (CREATE INDEX) en las columnas hospital_id, fecha y tipo_dieta de la tabla encuestas. Esto redujo el tiempo de respuesta de la base de datos de 400ms a apenas 15ms.
- **Auditoría de Seguridad (SecOps):**
  - *Revisión RLS:* Se auditaron nuevamente las políticas *Row Level Security*. Se escribieron pruebas manuales inyectando tokens JWT manipulados para asegurar de forma taxativa que es criptográficamente imposible que un hospital intercepte el flujo de datos de otro.
  - *Sanitización de Entradas:* Para evitar ataques de *Cross-Site Scripting (XSS)* en el campo de texto de "Observaciones" de la encuesta, se implementaron rutinas de limpieza en React que escapan caracteres peligrosos (<script>, <iframe>) antes de enviar el *payload* (carga útil) en formato JSON a Supabase.

20-febrero-2026 / Generación de Datos Simulados (*Mock Data*) y Pruebas de Estrés

- **Contexto y Objetivos:** Para poder defender el proyecto ante el tribunal, un Dashboard vacío o con 5 encuestas de prueba no demuestra el potencial real del aplicativo. Se necesitaba generar un volumen masivo de datos para realizar pruebas de estrés (*Stress Testing*).
- **Desarrollo Técnico:** * *Script de Seeding:* Se desarrolló un pequeño *script* auxiliar en Node.js utilizando librerías de generación aleatoria (como *Faker.js*). Este script se conectó a la API de Supabase y automatizó la inserción de **más de 1.500 encuestas ficticias**, distribuidas lógicamente con diferentes marcas de tiempo (desayunos a las 08:00, cenas a las 20:00), asignadas a distintas dietas y plantas de hospital.
  - *Pruebas de Estrés en Frontend:* Con la base de datos llena, se cargó el Dashboard del gestor. Se monitorizó el consumo de memoria RAM del navegador. Se optimizó la librería *Recharts* utilizando los *Hooks* de memorización de React (useMemo y useCallback) para evitar que la interfaz se congelara (cuello de botella en el hilo principal de JavaScript) al recalcular gráficas con miles de nodos de datos.
  - *Optimización de PDFs:* Se comprobó que la exportación intensiva de informes a formato PDF (mediante html2canvas y jsPDF) funcionaba de manera asíncrona, mostrando un *spinner* de carga al usuario sin bloquear la navegación web.

27-febrero-2026 / Cierre de Documentación y Manual de Despliegue

- **Contexto y Objetivos:** Garantizar el "Bus Factor" del proyecto; es decir, asegurar que si el desarrollador original no está, cualquier otro ingeniero o administrador de sistemas pueda entender, instalar y ejecutar el proyecto desde cero.
- **Desarrollo Técnico:** Semana dedicada íntegramente a la clausura documental.
  - Se redactó y empaquetó el **Manual de Despliegue**, estructurándolo como una receta paso a paso. Se detallaron los comandos exactos de terminal (git clone, npm install, npm run build), la estructura requerida del archivo .env.local, y los pasos para configurar el pipeline de CI/CD en la plataforma Vercel.
  - Se adjuntaron los volcados de código SQL (pg_dump) necesarios para recrear las tablas y políticas de Supabase.
  - Finalmente, se realizó una exhaustiva revisión ortotipográfica, de formato y de consistencia cruzada entre los cuatro manuales principales (Usuario, Técnico, Despliegue y Proyecto), asegurando que los diagramas UML coincidieran exactamente con el código final.

05-marzo-2026 / Congelación de Código (*Code Freeze*) y Versionado Semántico

- **Contexto y Objetivos:** *Hito de Entrega Documental:* "Cierre y congelación de la fase de análisis y diseño. Se entrega al tutor toda la documentación técnica (más de 30 páginas) que servirá como hoja de ruta inamovible."
- **Desarrollo Técnico:** * *Code Freeze:* Se decretó la congelación del repositorio. A partir de esta fecha, se aplicó una estricta política de "cero nuevas *features* (funcionalidades)". Cualquier *commit* en Git estuvo restringido única y exclusivamente a la corrección de errores críticos (*bugfixing*).
  - *Pruebas de Regresión:* Se ejecutaron pruebas de regresión manuales, recorriendo el "Camino Feliz" (*Happy Path*) de la aplicación: un paciente rellena la encuesta -> la base de datos se actualiza -> el gestor visualiza el cambio en la gráfica -> el gestor exporta el PDF.
  - *Release Final:* Tras comprobar que no existían errores bloqueantes, se etiquetó el código fuente en GitHub bajo la etiqueta de versionado semántico **v1.0.0**, marcando la versión definitiva estable (Release Candidate). Se forzó un último redespliegue en Vercel para asegurar que la nube reflejaba exactamente esta versión.

12-marzo-2026 / Preparación de la Defensa y Creación de Material Audiovisual

- **Contexto y Objetivos:** Un proyecto excelente puede fracasar si no se sabe comunicar. La penúltima semana se centró en técnicas de oratoria, presentación de producto e ingeniería de ventas aplicadas al TFG.
- **Desarrollo Técnico y Narrativo:** * Se estructuró el discurso de presentación (Pitch) en torno al valor estratégico que la tecnología aporta al modelo **COCINHEX**. Se preparó un esquema para justificar ante el tribunal por qué se tomaron decisiones como usar *React* en lugar de *Angular*, o un *BaaS* como *Supabase* en lugar de un servidor monolítico tradicional.
    - *Mitigación de Riesgos:* Las demostraciones en vivo (*Live Demos*) son propensas a fallar por problemas externos (caídas de Wi-Fi en el centro educativo). Para mitigar este riesgo, además de preparar el entorno real, se grabó y editó un vídeo en alta definición (Screencast) mostrando el flujo completo de la aplicación, garantizando que el tribunal pudiera evaluar el funcionamiento del software bajo cualquier circunstancia.

19-marzo-2026 / Entrega Definitiva y Defensa del Proyecto

- **Contexto y Objetivos:** Hito final del ciclo formativo. Consolidación de todo el trabajo académico y técnico de los dos años de Grado Superior.
- **Actividades Realizadas:** * Empaquetado final de los entregables: generación del archivo comprimido con el código fuente (excluyendo la pesada carpeta node_modules), volcado estructural de la base de datos en archivo .sql, y la generación en formato PDF de la carpeta oficial de Documentación.
  - Presentación oral y defensa pública del Trabajo Final de Grado **Hospifood Quality**. Se realizó la demostración empírica de la consecución de todos los objetivos funcionales y estratégicos planteados en septiembre, demostrando la viabilidad real de implementar este sistema en los hospitales de la red sanitaria extremeña.

4. Conclusiones

El desarrollo, conceptualización y futura implementación del sistema **Hospifood Quality** representa la culminación práctica de los conocimientos adquiridos durante el Grado Superior en Desarrollo de Aplicaciones Web (DAW). Tras agotar las fases de análisis, diseño arquitectónico y planificación estratégica documentadas en esta memoria, se pueden extraer las siguientes conclusiones fundamentales:

Consecución de Objetivos y Viabilidad Técnica

El objetivo principal del proyecto era resolver una deficiencia operativa real en el Servicio Extremeño de Salud (SES): la lentitud y el coste ecológico/administrativo del procesamiento en papel del método COCINHEX. A través de una metodología de trabajo *Design-First*, se ha demostrado que es técnicamente viable construir una solución 100% digital que elimine estas barreras. La adopción de una arquitectura *Serverless* (React + Supabase) ha probado ser la decisión más acertada, ya que permite a la administración pública desplegar software de altísimo rendimiento sin incurrir en costes prohibitivos de mantenimiento de servidores tradicionales.

Innovación en la Experiencia de Usuario (UX) e Inclusión Digital

Uno de los mayores logros analíticos de este proyecto ha sido la resolución del "reto del paciente". Al diseñar una interfaz basada en el principio *"Fat Finger"* y utilizar iconografía universal (caritas de valoración) en lugar de formularios web clásicos, Hospifood Quality rompe la brecha digital. Se ha garantizado que cualquier paciente ingresado, independientemente de su edad, nivel de

alfabetización digital o condición motora, pueda ejercer su derecho a evaluar el servicio de alimentación de forma autónoma, anónima y en menos de un minuto.

Robustez, Ciberseguridad y Cumplimiento Normativo

Desde el punto de vista de la ingeniería de software, el proyecto destaca por su enfoque *"Security by Design"* (Seguridad desde el Diseño). Al delegar la autorización y el filtrado de datos a la capa más profunda de la base de datos mediante Row Level Security (RLS) en PostgreSQL, se ha blindado el sistema contra fugas de información. Se ha garantizado el cumplimiento estricto del RGPD, asegurando que el aislamiento de datos en este entorno *Multi-Tenant* (múltiples hospitales usando la misma app) sea criptográficamente seguro.

Impacto Estratégico para la Administración Pública

La puesta en marcha de esta herramienta, transformará datos brutos en inteligencia de negocio (*Business Intelligence*). Los gestores de calidad y responsables de cocina ya no tendrán que esperar semanas para tabular encuestas; dispondrán de un *Dashboard* en tiempo real que les permitirá reaccionar de forma inmediata ante una caída en la calidad del servicio (por ejemplo, fallos en la cadena de frío de los alimentos), mejorando directamente el bienestar y la recuperación del paciente hospitalizado.

Valoración Personal y Académica

A nivel formativo, este proyecto ha supuesto un reto mayúsculo que va mucho más allá de la simple escritura de código. Ha exigido asumir roles de consultor tecnológico, analista de datos, diseñador UI/UX y arquitecto *Cloud*. La elaboración de más de 40 páginas de documentación técnica y funcional

demuestra una madurez profesional que asegura que la fase de programación (marzo-junio) se ejecutará sobre cimientos sólidos, minimizando la improvisación y maximizando la calidad del software entregable.

Líneas de Trabajo Futuro

Aunque el Producto Mínimo Viable (MVP) definido en esta memoria cubre todas las necesidades actuales del método COCINHEX, el sistema ha sido diseñado con una arquitectura escalable que permite futuras iteraciones (v2.0). Como líneas de trabajo futuro se proponen:

1. **Integración HL7/FHIR:** Conectar la aplicación con los sistemas de admisión del SES para que la tablet de la habitación sepa automáticamente qué dieta tiene el paciente, sin pedirle ningún dato.
2. **Inteligencia Artificial (Machine Learning):** Implementar algoritmos predictivos en el *Dashboard* que avisen al hospital con antelación si, por tendencias históricas, se prevé una caída en la satisfacción general durante los fines de semana o cambios de turno.
3. **Despliegue Multiplataforma:** Convertir la aplicación web en una PWA (Progressive Web App) con capacidades *offline*, garantizando que las encuestas se guarden en el dispositivo incluso si hay caídas en la red Wi-Fi del hospital.

En definitiva, **Hospifood Quality** no es solo un ejercicio académico, sino un producto de software real, auditable y escalable, preparado para generar un impacto positivo y tangible en la sanidad pública de la región.